

Bài tập chương IV

Câu 1:

a. Định nghĩa khái niệm phối hợp và đồng bộ trong các hệ thống phân tán

Phối hợp (Coordination)

Phối hợp trong hệ thống phân tán là **sự kết nối hoạt động giữa các tiến trình để cùng nhau giải quyết một nhiệm vụ chung**. Nó liên quan đến việc **quản lý các tương tác và sự phụ thuộc lẫn nhau giữa các tiến trình**, để đảm bảo rằng toàn bộ hệ thống có thể vận hành như một thể thống nhất.

Đồng bộ (Synchronization)

Đồng bộ là quá trình các tiến trình **thực hiện các hoạt động có cùng chu kỳ hoặc cùng tốc độ, trong cùng một thời gian**. Đồng bộ tạo ra **sự phối hợp nhịp nhàng**, trong đó **một tiến trình có thể phải chờ tiến trình khác hoàn thành nhiệm vụ trước khi tiếp tục**.

Sự khác nhau chính giữa phối hợp và đồng bộ

- **Phối hợp** là khái niệm tổng quát hơn, tập trung vào việc **tổ chức hoạt động giữa các tiến trình** sao cho chúng hướng tới một mục tiêu chung.
- **Đồng bộ** là một phần trong quá trình phối hợp, tập trung vào **việc điều chỉnh thời điểm thực hiện các hành động** để tránh xung đột hoặc đảm bảo thứ tự.

Tóm lại: Phối hợp nói về "làm việc cùng nhau", còn đồng bộ là "làm việc đúng thời điểm với nhau".

b. Tầm quan trọng của việc đồng bộ hóa thời gian trong hệ thống phân tán

Trong hệ thống phân tán, mỗi máy có **đồng hồ vật lý riêng**, và **không có gì đảm bảo tất cả đồng hồ sẽ có cùng thời gian tại một thời điểm**. Sự khác biệt này gây ra nhiều vấn đề nghiêm trọng, nhất là khi các tiến trình cần hoạt động theo thứ tự hoặc cần ghi nhận sự kiện chính xác về thời gian.

Ví dụ minh họa: Hai lập trình viên cùng chỉnh sửa mã nguồn

Giả sử **hai lập trình viên cùng làm việc trên một tệp mã nguồn đặt trên hệ thống phân tán**, mỗi người trên một máy khác nhau:

- Lập trình viên A lưu tệp lúc 10:00 (theo đồng hồ máy A).
- Lập trình viên B cũng lưu cùng tệp lúc 09:58 (theo đồng hồ máy B, nhưng thực tế là **sau** lập trình viên A).

Nếu hệ thống **không đồng bộ thời gian giữa các máy**, hệ thống có thể **coi phiên bản của B là mới hơn**, dẫn đến việc **ghi đè mất dữ liệu của A**.

Hậu quả khi đồng hồ không khớp nhau:

- **Sai lệch dữ liệu:** Phiên bản cũ có thể ghi đè phiên bản mới.
- **Xung đột không phát hiện được:** Các tiến trình xử lý thông tin dựa trên thời gian sẽ ra quyết định sai.
- **Thứ tự sai lệch:** Các thông điệp gửi/nhận bị xử lý sai thứ tự, gây lỗi logic trong các ứng dụng phân tán.

Tại sao cần đồng bộ thời gian trước khi giải quyết các vấn đề khác?

- Vì **thời gian là cơ sở** để đánh giá sự kiện nào xảy ra trước, sau.
- Nếu không đồng bộ được thời gian, thì các giải pháp như **giải quyết tương tranh tài nguyên, ghi log, quyết định ai được quyền truy cập**, v.v. đều trở nên **thiếu chính xác hoặc không hiệu quả**.

c. Nguyên tắc hoạt động của giải thuật Cristian

Giải thuật Cristian là một phương pháp đồng bộ thời gian giữa máy khách và máy chủ thời gian (NTP server) trong hệ thống phân tán, do Cristian đề xuất năm 1989. Mục tiêu là giúp đồng hồ trên máy khách được điều chỉnh theo thời gian thực UTC từ máy chủ, đồng thời hạn chế sai lệch do **độ trễ mạng** gây ra.

Giải thuật hoạt động như sau:

1. **Tại thời điểm T1**, máy khách gửi một yêu cầu đồng bộ đến máy chủ thời gian.
2. **Máy chủ nhận yêu cầu tại thời điểm T2**, sau đó chuẩn bị và gửi phản hồi kèm thời gian thực UTC **tại thời điểm T3**.
3. **Máy khách nhận phản hồi tại thời điểm T4**.

Giả sử độ trễ hai chiều là gần bằng nhau (gửi và nhận), độ lệch thời gian (θ – theta) giữa đồng hồ máy khách và máy chủ được tính bằng công thức:

$$\theta = \frac{(T2 - T1) + (T3 - T4)}{2}$$

Máy khách sẽ cộng thêm giá trị θ vào đồng hồ của mình để điều chỉnh lại cho gần đúng với máy chủ.

Cách ước lượng độ trễ mạng (δ – delta)

Công thức tính độ trễ mạng mỗi chiều:

$$\delta = \frac{(T4 - T1) - (T3 - T2)}{2}$$

Giá trị này giúp ước lượng độ trễ mạng thực tế, từ đó có thể lọc ra giá trị độ lệch (θ) có độ tin cậy cao nhất. Trong thực tế, thuật toán thực hiện nhiều lần (ví dụ 8 lần) để lấy trung bình hoặc chọn θ ứng với δ nhỏ nhất làm độ lệch chính xác.

Xử lý các trường hợp đặc biệt khi hiệu chỉnh đồng hồ

- Nếu $\theta < 0$: đồng hồ máy khách đang nhanh $\rightarrow$ **không nên lùi thời gian** vì có thể gây nghịch lý thời gian (sự kiện xảy ra sau lại có timestamp nhỏ hơn).
 - Cách xử lý: tạo **vòng lặp hiệu chỉnh dần** bằng cách ngưng tăng đồng hồ trong những chu kỳ nhỏ (ví dụ 10 ms) cho đến khi thời gian được đồng bộ hoàn toàn.
- Nếu $\theta > 0$: đồng hồ máy khách chậm hơn $\rightarrow$ có thể tăng thời gian, nhưng **phải kiểm tra các tác vụ theo lịch**, tránh bị **bỏ qua nhiệm vụ quan trọng**.

d. Giải thuật Berkeley: Mô tả cách thức hoạt động

Giải thuật **Berkeley**, do **Gusella và Zatti** phát triển năm **1989** tại Đại học California, là một giải pháp **đồng bộ thời gian trong hệ thống phân tán nội bộ**, không cần truy cập đến máy chủ thời gian quốc tế như UTC.

Nguyên lý hoạt động:

1. **Tiến trình điều phối (coordinator)** — một trong các máy tính trong nhóm (cũng là thành viên) — gửi thông điệp chứa **thời gian hiện tại của nó** cho tất cả các **máy**

khách trong hệ thống.

2. **Mỗi máy khách** nhận thông điệp và tính **độ lệch thời gian** bằng cách lấy thời gian của chính nó **trừ đi** thời gian của máy điều phối đã gửi đến.
3. Sau khi nhận tất cả các độ lệch phản hồi từ các máy khách, **máy điều phối** tính **giá trị trung bình** của các độ lệch này (có thể loại bỏ ngoại lệ nếu cần), rồi:
 - **Tự điều chỉnh** đồng hồ của mình theo độ lệch trung bình.
 - Tính toán **độ lệch cần điều chỉnh cho từng máy** và gửi lại thông điệp yêu cầu điều chỉnh thời gian.
4. **Mỗi máy** sau đó điều chỉnh thời gian của mình theo yêu cầu điều phối.

Ví dụ trong tài liệu: nếu thời gian hiện hành là 03:00, các độ lệch là {0, -10, 15}, trung bình là +5 phút. Điều phối viên cộng thêm 5 phút, máy bên trái cộng thêm 15 phút, máy bên phải điều chỉnh lùi 20 phút.

Lưu ý kỹ thuật:

- Độ trễ mạng và thời gian xử lý cần được **ước lượng** để tránh sai lệch lớn.
- Nếu máy điều phối bị lỗi, cần thực hiện **bầu chọn** lại máy mới.
- **Tính nhất quán** nội bộ cao, nhưng **không đảm bảo đúng theo UTC**.

So sánh giải thuật Berkeley với giải thuật Cristian

Tiêu chí	Giải thuật Cristian	Giải thuật Berkeley
Nguồn thời gian	Dựa trên máy chủ NTP cung cấp thời gian chuẩn UTC.	Không có thời gian chuẩn , chỉ đồng bộ theo thỏa thuận nội bộ giữa các thành viên.
Vai trò máy chủ	Thụ động : chỉ phản hồi khi có yêu cầu từ máy khách.	Chủ động : tiến hành điều phối thu thập thông tin, tính toán và yêu cầu các thành viên điều chỉnh.
Vai trò máy khách	Chủ động : gửi yêu cầu, nhận thời gian phản hồi, tự tính toán độ lệch.	Thụ động : chỉ phản hồi độ lệch thời gian và thực hiện điều chỉnh theo yêu cầu điều phối.
Cơ chế đồng bộ	Một chiều: từ máy chủ về máy khách.	Hai chiều: từ điều phối → máy khách → điều phối → máy khách.

Độ chính xác	Có thể đạt độ chính xác cao (~50ms) nếu hạ tầng ổn định và đo trễ chính xác.	Phụ thuộc nội bộ nhóm , có thể bị ảnh hưởng bởi trễ mạng và thời gian phản hồi.
Khả năng thích ứng	Phù hợp cả mạng LAN và Internet (dùng NTP), dễ mở rộng.	Chỉ phù hợp mạng nội bộ , khó mở rộng hoặc duy trì nếu có nhiều thay đổi thành viên.
Độ tin cậy	Phụ thuộc vào độ tin cậy của máy chủ thời gian NTP.	Phụ thuộc vào độ tin cậy của thành viên điều phối và các máy trong nhóm.
Khả năng phục hồi	Nếu máy chủ mất kết nối, không thể đồng bộ.	Nếu điều phối viên bị lỗi, có thể chọn máy khác thông qua giải thuật bầu chọn .
Rủi ro bảo mật	Tốt hơn do dùng giao thức chuẩn như NTPv4 (RFC-5905).	Bảo mật kém , dễ bị thao túng bởi thành viên độc hại (không có biện pháp xác thực).
Ứng dụng phù hợp	Hệ thống yêu cầu thời gian chính xác, hoạt động ổn định và kết nối Internet.	Mạng nội bộ, hệ thống không kết nối Internet hoặc không cần đồng bộ theo UTC.

Ưu điểm và nhược điểm từng giải thuật

Giải thuật Cristian

- **Ưu điểm:**
 - Độ chính xác cao nếu dùng NTP chuẩn.
 - Đơn giản, dễ triển khai.
 - Có thể áp dụng cho các hệ thống Internet.
- **Nhược điểm:**
 - Phụ thuộc hoàn toàn vào máy chủ NTP.
 - Không thể quay lui đồng hồ máy khách (cần điều chỉnh dần).
 - Gặp khó khăn nếu máy khách không thể tiếp cận máy chủ thời gian.

Giải thuật Berkeley

- **Ưu điểm:**

- Không phụ thuộc thời gian chuẩn UTC.
- Có thể dùng trong mạng nội bộ không kết nối Internet.
- Dễ chọn lại điều phối viên nếu máy điều phối cũ bị lỗi.

- **Nhược điểm:**

- Sai số có thể lớn nếu có độ trễ mạng cao hoặc phản hồi không đồng bộ.
- Bảo mật thấp (dễ bị máy giả mạo ảnh hưởng).
- Không phù hợp với môi trường có yêu cầu thời gian chuẩn xác cao theo UTC.

e. Giải thuật trung bình và tham chiếu quảng bá: Hãy giải thích nguyên tắc hoạt động của giải thuật đồng bộ trung bình và giải thuật đồng bộ tham chiếu quảng bá trong hệ thống phân tán. Nêu những ưu điểm và nhược điểm chính của từng giải thuật.

1. Giải thuật đồng bộ Trung bình

❖ Nguyên tắc hoạt động:

- Không có **thành viên điều phối** như thuật toán Berkeley.
- Mỗi thành viên trong hệ thống:
 1. **Gửi (quảng bá)** thời gian hiện tại của mình vào đầu mỗi chu kỳ đồng bộ.
 2. **Lắng nghe** thời gian từ các thành viên khác trong khoảng thời gian ngắn SSS.
 3. Thu thập các thời gian nhận được và **loại bỏ** một số giá trị lớn nhất và nhỏ nhất để **tránh nhiễu**.
 4. Tính **giá trị trung bình** của các thời gian còn lại và điều chỉnh đồng hồ của mình theo giá trị trung bình đó.

Công thức tính thời gian mới (nếu loại nhiễu): Lấy trung bình các giá trị còn lại sau khi loại bỏ biên.

Ưu điểm:

- **Không cần thành viên điều phối**, nên không có điểm thất bại trung tâm.

- Có khả năng **lọc nhiễu** bằng cách loại bỏ các giá trị biên.
- Thích hợp trong **mạng có độ tin cậy cao** và máy tính thường xuyên trực tuyến.

Nhược điểm:

- **Không đồng bộ hoàn toàn chính xác**, do độ trễ mạng và trôi thời gian giữa các máy.
- Nếu thời gian lắng nghe SSS quá ngắn hoặc chênh lệch thời gian ban đầu quá lớn → **một số thành viên có thể không được đồng bộ**.
- Dễ bị ảnh hưởng nếu có quá nhiều máy gửi dữ liệu sai hoặc nhiễu.

2. Giải thuật đồng bộ Tham chiếu quảng bá

❖ Nguyên tắc hoạt động:

- Phù hợp với **mạng không dây**, nơi độ trễ truyền thông thường **không ổn định**.
- Một thành viên **gửi tín hiệu đồng bộ đơn giản (không chứa thời gian)** gọi là **thông điệp m**.
- Các thành viên khác khi **nhận được tín hiệu m**, sẽ:
 - Ghi lại thời gian nội bộ tại thời điểm nhận.
 - Gửi thời gian này đến các thành viên khác để tính toán **độ lệch thời gian giữa từng cặp**.

Ưu điểm:

- Phù hợp với **mạng không dây**, nơi độ trễ khó kiểm soát.
- **Không cần trao đổi hai chiều** → Giảm độ trễ không đoán trước.
- Giảm tải và tiết kiệm năng lượng trong **các mạng cảm biến**.
- Độ trễ lan truyền vật lý gần như **xấp xỉ hằng số** → dễ kiểm soát.

Nhược điểm:

- **Không đồng bộ tuyệt đối** vì bỏ qua thời gian xử lý nội bộ của từng máy.
- Cần **nhiều lần truyền thông điệp** để đạt kết quả chính xác.
- **Không đồng bộ theo thời gian chuẩn UTC**, chỉ phù hợp cho **đồng bộ nội bộ**.

Câu 2:

a. Vì sao việc đồng bộ hóa thời gian vật lý trong hệ thống phân tán gặp nhiều khó khăn?

Việc đồng bộ thời gian vật lý trong hệ thống phân tán **gặp nhiều khó khăn** do những lý do sau:

1. Mỗi máy tính có đồng hồ riêng biệt:

- Trong hệ thống phân tán, mỗi thành viên sử dụng một **đồng hồ vật lý riêng**. Không có gì đảm bảo rằng tất cả các đồng hồ này **đếm thời gian giống nhau**, do ảnh hưởng từ tốc độ đếm, độ lệch pha (skew), hay sai số trôi (drift).

2. Độ trễ mạng không đồng nhất:

- Thời gian truyền thông điệp qua mạng **phụ thuộc vào độ trễ mạng**, có thể biến đổi bất ngờ, gây sai lệch lớn giữa thời gian thực và thời gian nhận/gửi thông điệp.

3. Thông điệp có thể đến không đúng thứ tự:

- Mạng máy tính không đảm bảo **thứ tự phân phát thông điệp**, dẫn đến tình trạng các tiến trình nhận thông điệp **không theo đúng trình tự thời gian**, gây rối loạn thứ tự sự kiện.

4. Không có đồng hồ toàn cục:

- Trong hệ phân tán, **không tồn tại một đồng hồ toàn cục chung** cho toàn hệ thống. Vì vậy, rất khó để xác định chính xác **thứ tự xảy ra của các sự kiện** tại các máy khác nhau.

5. Tính đồng bộ theo thời gian thực là không khả thi tuyệt đối:

- Đối với **ứng dụng thời gian thực**, yêu cầu độ chính xác cao về thời điểm xử lý, nhưng việc **đồng bộ đồng hồ vật lý tuyệt đối là bất khả thi** trong môi trường phân tán.

Khái niệm Đồng hồ Logic của Lamport

Để giải quyết vấn đề trên, **Lamport** đã đề xuất một giải pháp thay thế gọi là **đồng hồ logic** – một cách tiếp cận không phụ thuộc vào đồng hồ vật lý mà dựa trên **quan hệ nhân quả giữa các sự kiện**.

❖ Nguyên lý chính:

- Dựa trên khái niệm **“xảy ra trước”** ($\rightarrow$):

- Nếu **a và b** là hai sự kiện trên cùng một tiến trình và **a xảy ra trước b**, thì $a \rightarrow b$.
- Nếu **a là gửi một thông điệp**, và **b là nhận thông điệp đó**, thì $a \rightarrow b$.
- Quan hệ xảy ra trước có tính **bắc cầu**: nếu $a \rightarrow b$ và $b \rightarrow c$, thì $a \rightarrow c$.

❖ **Đồng hồ logic hoạt động như sau:**

- Mỗi tiến trình P_i duy trì một **bộ đếm logic** C_i .
- Trước khi một tiến trình thực hiện sự kiện, nó **tăng bộ đếm thêm 1 đơn vị**: $C_i \leftarrow C_i + 1$.
- Khi gửi thông điệp m , nó **gắn nhãn thời gian** $ts(m) = C_i$ vào thông điệp.
- Khi tiến trình P_j nhận thông điệp m :
 - Nó cập nhật bộ đếm logic: $C_j \leftarrow \max(C_j, ts(m)) + 1$.

❖ **Vai trò:**

- Đảm bảo rằng nếu **một sự kiện a xảy ra trước sự kiện b**, thì $C(a) < C(b)$.
- Cho phép **xác định thứ tự tương đối** giữa các sự kiện mà **không cần phụ thuộc vào thời gian vật lý**.
- Là nền tảng cho việc **đồng bộ hóa hành vi** giữa các tiến trình trong hệ thống phân tán.

b. Định nghĩa quan hệ “xảy ra trước” (happened-before) trong đồng hồ logic Lamport

Trong hệ thống phân tán, do **không có đồng hồ vật lý chung** và độ trễ mạng không ổn định, việc xác định thứ tự xảy ra của các sự kiện giữa các tiến trình khác nhau là một thách thức. Để giải quyết vấn đề này, **Lamport** đã đề xuất khái niệm **quan hệ xảy ra trước (happened-before)** để thay thế khái niệm thời gian vật lý, giúp xác định thứ tự hợp lý giữa các sự kiện.

Định nghĩa quan hệ xảy ra trước ($\rightarrow$)

Quan hệ **xảy ra trước** giữa hai sự kiện được định nghĩa như sau:

1. Trên cùng một tiến trình:

- Nếu **a** và **b** là hai sự kiện xảy ra trên cùng một tiến trình, và **a xảy ra trước b**, thì ta nói $a \rightarrow b$.
- Ví dụ: Một tiến trình thực hiện **gửi dữ liệu** rồi mới **ghi log** thì gửi dữ liệu xảy ra trước.

2. Giữa hai tiến trình khác nhau (thông điệp):

- Nếu **a** là sự kiện **gửi một thông điệp**, và **b** là sự kiện **nhận đúng thông điệp đó**, thì $a \rightarrow b$.
- Đây là quan hệ nhân quả rõ ràng, vì muốn nhận được thông điệp thì bắt buộc phải có hành động gửi trước đó.

Các tính chất của quan hệ xảy ra trước

Tính chất bắc cầu:

- Nếu $a \rightarrow b$ và $b \rightarrow c$, thì $a \rightarrow c$.
- Tức là: nếu một sự kiện dẫn đến một sự kiện khác, và sự kiện đó lại dẫn đến một sự kiện khác nữa, thì sự kiện đầu tiên cũng là nguyên nhân của sự kiện cuối.

Tính tương tranh (Concurrency):

- Nếu **x** và **y** là hai sự kiện xảy ra trên hai tiến trình khác nhau, và **không có mối quan hệ nhân quả nào xác định được giữa chúng**, thì ta nói **x và y là các sự kiện tương tranh**.
- Ký hiệu: $x \parallel y$
- Điều này có nghĩa là hai sự kiện **có thể xảy ra đồng thời**, hoặc theo bất kỳ thứ tự nào, tùy thuộc vào môi trường thực thi và độ trễ mạng.

c. 1. Cơ chế gán nhãn thời gian Lamport

Thuật toán Lamport giúp xác định thứ tự các sự kiện trong hệ thống phân tán bằng cách dùng **đồng hồ logic** chứ không phải đồng hồ vật lý. Mỗi tiến trình có một **bộ đếm thời gian logic riêng** (ký hiệu là C_i với tiến trình P_i) và thực hiện như sau:

✓ Khi có sự kiện nội bộ:

- Tiến trình P_i **tự tăng bộ đếm**:

$$C_i \leftarrow C_i + 1$$

✓ Khi gửi thông điệp:

- Tăng bộ đếm như sự kiện nội bộ:
 $C_i \leftarrow C_i + 1$
- Gửi thông điệp kèm theo nhãn thời gian:
 $ts(m) = C_i$

✓ Khi nhận thông điệp:

- Tăng bộ đếm của tiến trình nhận:
 $C_j \leftarrow C_j + 1$
- **Điều chỉnh đồng hồ** theo nhãn thời gian nhận được:
 $C_j \leftarrow \max(C_j, ts(m)) + 1$
(nếu đồng hồ hiện tại nhỏ hơn nhãn thời gian trong thông điệp).

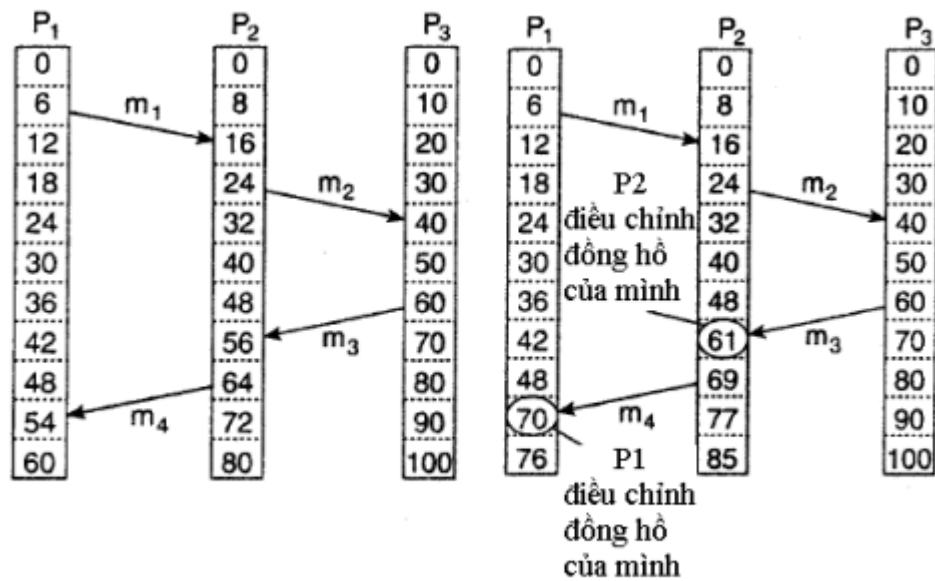
Cơ chế điều chỉnh khi nhãn thời gian đến nhỏ hơn thời điểm đã gửi

Trong hệ thống phân tán, **không đảm bảo thông điệp đến đúng thứ tự**, và đồng hồ logic của các tiến trình **tăng không đồng đều**. Vì vậy có thể xảy ra tình huống một thông điệp được nhận **trước thời điểm gửi**, nếu nhìn theo giá trị đồng hồ logic. Điều này **vi phạm quan hệ nhân quả** ($a \rightarrow b \Rightarrow C(a) < C(b)$), nên cần điều chỉnh.

✓ Cơ chế điều chỉnh:

- Nếu một thông điệp đến với nhãn thời gian **nhỏ hơn hoặc bằng** thời gian gửi $\rightarrow$ **tiến trình nhận cần tăng nhãn thời gian lên ít nhất 1 đơn vị so với thời điểm gửi**.

- Cụ thể:
Nếu $C_j < ts(m) \rightarrow C_j = ts(m) + 1$



Hình 4.6 Đồng bộ nhãn thời gian Lamport

Giải thích

Giai đoạn 1:

- Ban đầu, đồng hồ của các tiến trình P1, P2, P3 đều bắt đầu từ 0.
- Các tiến trình đếm với nhịp khác nhau:
 - P1 mỗi lần +6
 - P2 mỗi lần +8
 - P3 mỗi lần +10

m1: Từ P1 → P2

- P1 gửi m1 tại thời điểm 6 → $ts(m1) = 6$
- P2 nhận tại thời điểm 16
→ Không cần điều chỉnh ($16 > 6$)

m2: Từ P2 → P3

- Gửi lúc 32, nhận lúc 48
→ Không cần điều chỉnh

m3: Từ P3 → P2

- Gửi lúc 60, nhận lúc 56
→ **Có vấn đề** vì $56 < 60$

Điều chỉnh:

- Vì $56 < 60 \rightarrow$ P2 điều chỉnh:
 $C(P2) = 60 + 1 = 61$

m4: Từ P2 → P1

- P2 gửi tại thời điểm 69
- P1 nhận lúc 54 (nhỏ hơn 69)
→ P1 điều chỉnh:
 $C(P1) = 69 + 1 = 70$

Mục đích của việc điều chỉnh

Việc điều chỉnh như trên nhằm đảm bảo rằng:

Nếu một sự kiện **a** xảy ra trước sự kiện **b**, thì luôn luôn $C(a) < C(b)$

Điều này duy trì tính **nhân quả** của các sự kiện trong hệ thống phân tán, bất chấp sự chênh lệch đồng hồ giữa các tiến trình hoặc độ trễ mạng.

d. So sánh Đồng hồ Lamport và Đồng hồ Vector

Tiêu chí	Đồng hồ Lamport	Đồng hồ Vector
Khả năng xác định quan hệ nhân quả	Không đầy đủ: chỉ đảm bảo nếu $a \rightarrow b$ thì $C(a) < C(b)$, nhưng $C(a) < C(b)$ không ngụ ý $a \rightarrow b$.	Đầy đủ: nếu $VC(a) < VC(b)$ thì chắc chắn $a \rightarrow b$, và có thể phát hiện khi hai sự kiện tương tranh .
Cấu trúc dữ liệu	Một số nguyên đơn giản (<i>số đếm</i>)	Một vector gồm n phần tử, mỗi phần tử tương ứng với một tiến trình
Kích thước dữ liệu	Nhỏ (1 giá trị)	Lớn hơn (vector có kích thước bằng số tiến trình)
Độ chính xác trong biểu diễn nhân quả	Thấp	Cao – phản ánh chính xác lịch sử sự kiện của toàn hệ thống

Khả năng sắp xếp sự kiện

Có thể sắp xếp tuyến tính nhưng không phản ánh được quan hệ nhân quả thực sự

Có thể sắp xếp chính xác theo quan hệ nhân quả, bao gồm cả phát hiện sự kiện tương tranh

Nguyên tắc hoạt động của Đồng hồ Vector

Cấu trúc dữ liệu:

- Mỗi tiến trình P_i duy trì một **vector** VC_i kích thước n (với n là số lượng tiến trình).
- Mỗi phần tử $VC_i[j]$ thể hiện số lượng sự kiện mà tiến trình P_i biết đã xảy ra tại tiến trình P_j .

Cập nhật đồng hồ vector:

1. Sự kiện nội tại hoặc gửi thông điệp:

- Tăng phần tử tương ứng với chính tiến trình đó:

$$VC_i[i] \leftarrow VC_i[i] + 1$$

- Nếu gửi thông điệp m , đính kèm nhãn thời gian vector $ts(m) = VC_i$.

2. Nhận thông điệp m từ tiến trình P_k :

- Tăng phần tử tương ứng với tiến trình nhận:

$$VC_j[j] \leftarrow VC_j[j] + 1$$

- Gộp vector nhận được từ thông điệp:

$$VC_j[l] \leftarrow \max(VC_j[l], ts(m)[l]) \quad \text{với mọi } l = 1 \dots n$$

Điều kiện để chuyển thông điệp lên tầng ứng dụng (Causal Delivery Rule)

Điều kiện để chuyển thông điệp lên tầng ứng dụng (Causal Delivery Rule)

Khi tiến trình P_j nhận được thông điệp m từ P_i kèm nhãn thời gian $ts(m)$, thì thông điệp chỉ được chuyển lên tầng ứng dụng nếu đồng thời thỏa mãn hai điều kiện sau:

1. Thông điệp đến đúng thứ tự mong đợi từ P_i :

$$ts(m)[i] = VC_j[i] + 1$$

→ Tức là đây là thông điệp kế tiếp mà P_j mong nhận từ P_i .

2. Tất cả các thông điệp nhân quả trước đã được xử lý:

$$\forall k \neq i, ts(m)[k] \leq VC_j[k]$$

→ Nghĩa là P_j đã xử lý hết các sự kiện mà P_i biết tại thời điểm gửi thông điệp m .

=> Nếu một trong hai điều kiện không thỏa mãn, thông điệp phải **lưu vào vùng đệm** để xử lý sau.

e. 1. Trạng thái toàn cục trong hệ thống phân tán là gì?

Trạng thái toàn cục (Global State) của một hệ thống phân tán tại một thời điểm là tập hợp gồm:

- **Trạng thái cục bộ** của **tất cả các tiến trình** trong hệ thống (gồm các biến, chương trình đang chạy,...).
- **Trạng thái của các kênh truyền**, tức là **các thông điệp đang “giữa đường”**, đã được gửi nhưng chưa được nhận.

Do các tiến trình không chia sẻ đồng hồ chung và không có khái niệm thời gian tuyệt đối nên việc xác định trạng thái toàn cục chính xác là **rất khó** trong thực tế.

2. Khái niệm lát cắt (cut) là gì?

Lát cắt = tập hợp các mốc trạng thái (điểm ghi nhận) của tất cả các tiến trình tại một thời điểm tương đối, dùng để tái cấu trúc lại trạng thái hệ thống tại thời điểm đó.

Câu 3: (4)

a. Khái niệm “giải thuật bầu chọn” trong hệ thống phân tán và mục đích chính của nó

Khái niệm:

Giải thuật bầu chọn (Election Algorithm) là một cơ chế trong hệ thống phân tán nhằm **lựa chọn ra một tiến trình duy nhất đóng vai trò điều phối (coordinator)** trong hệ thống. Vai trò của điều phối viên là quản lý, giám sát hoặc điều phối các hoạt động dùng chung, chẳng hạn như phân chia tài nguyên, đồng bộ hóa hoặc thực hiện các nhiệm vụ đặc biệt.

Mục đích chính:

- Bầu chọn một **tiến trình điều phối mới** khi tiến trình điều phối hiện tại **gặp lỗi hoặc dừng hoạt động**.
- **Đảm bảo duy trì hoạt động ổn định** của hệ thống khi có sự cố xảy ra.
- **Tránh xung đột** và tình trạng nhiều tiến trình cùng điều phối.

b. Nguyên lý hoạt động cơ bản của giải thuật nổi bọt (Bully Algorithm)

Giải thuật **nổi bọt** (bully) được **Garcia-Molina** đề xuất năm 1982, với nguyên lý cơ bản là: **tiến trình có định danh cao nhất sẽ được chọn làm điều phối viên**.

Nguyên lý hoạt động tổng quát:

1. Khởi động quá trình bầu chọn:

- Một tiến trình Pk phát hiện điều phối viên hiện tại không còn phản hồi.
- Nó sẽ gửi thông điệp "**Bầu chọn**" đến tất cả các tiến trình có định danh cao hơn.

2. Xử lý thông điệp "**Bầu chọn**":

- Các tiến trình nhận được thông điệp "**Bầu chọn**" từ một tiến trình có định danh thấp hơn, nếu còn hoạt động:
 - Gửi lại thông điệp "**Không chấp nhận**" → báo rằng "Tôi còn sống và tôi mạnh hơn".
 - Đồng thời tự khởi tạo quá trình bầu chọn riêng.

3. Từ bỏ tranh cử:

- Tiến trình Pk nếu **nhận được ít nhất một thông điệp "Không chấp nhận"** thì **từ bỏ vai trò ứng cử viên**.

4. Trở thành điều phối viên:

- Nếu không nhận được bất kỳ phản hồi nào (các tiến trình có ID cao hơn đều đã chết), Pk tự **xưng là điều phối viên** và **gửi thông báo đến toàn hệ thống**.

5. Sự sống lại của điều phối viên cũ:

- Nếu tiến trình điều phối cũ (định danh cao hơn) sống lại sau khi bầu chọn hoàn tất, **có thể dẫn đến xung đột**, và đây là **lỗi hồng** cần giải pháp bổ sung.

c. Cho hệ thống gồm 6 tiến trình với định danh {P0,...,P5}, giả sử P2 phát hiện coordinator P5 lỗi, hãy mô tả chi tiết các bước thông điệp (và thứ tự) của giải thuật vòng (Ring) để chọn ra tiến trình mới.

Giả định ban đầu:

- Các tiến trình được sắp xếp trên một **vòng tròn logic** như sau:
 $P0 \rightarrow P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P5 \rightarrow P0 \rightarrow \dots$

- Tiến trình P2 phát hiện P5 lỗi và khởi động giải thuật bầu chọn.
- Tất cả các tiến trình từ P0 đến P4 còn hoạt động.
- Mục tiêu: Tìm ra tiến trình có định danh cao nhất còn sống, rồi chọn nó làm coordinator mới.

Bước 1: P2 khởi xướng bầu chọn

- P2 tạo thông điệp Election = [2]
- Gửi đến tiến trình kế tiếp là P3

Bước 2: Thăm vòng và thu thập định danh còn sống

- P3 nhận [2], thêm định danh của mình vào → [2, 3], gửi cho P4
- P4 nhận [2, 3], thêm định danh của mình vào → [2, 3, 4], gửi cho P5
- P5 đã lỗi, không phản hồi. P4 sẽ chờ trong timeout, rồi chuyển thông điệp đến tiến trình kế tiếp là P0
- P0 nhận [2, 3, 4], thêm định danh của mình vào → [2, 3, 4, 0], gửi đến P1
- P1 nhận [2, 3, 4, 0], thêm mình vào → [2, 3, 4, 0, 1], gửi đến P2
- P2 nhận lại thông điệp → kết thúc vòng bầu chọn

Bước 3: P2 xác định coordinator mới

- Tập các tiến trình còn sống: [0, 1, 2, 3, 4]
- Coordinator mới là tiến trình có định danh cao nhất: P4

Bước 4: Thông báo kết quả bầu chọn

- P2 tạo thông điệp Coordinator = [4]
- Gửi đến tiến trình kế tiếp P3
- Các bước chuyển tiếp:
 - P3 → P4 → (bỏ qua P5 vì lỗi) → P0 → P1 → P2

- Tất cả tiến trình nhận được thông báo rằng **P4 là điều phối viên mới**

d. 1. So sánh tổng quát theo 3 tiêu chí chính

Tiêu chí	Bully (Nổi bọt)	Ring (Vòng)	Wireless (Vasudevan 2004)
Khả năng chịu lỗi	Trung bình	Tốt	Rất tốt (dành riêng cho mạng không ổn định)
Số lượng thông điệp trao đổi	$O(n^2)$ (trong trường hợp xấu nhất)	$O(n)$	$O(n)$
Yêu cầu kiến trúc mạng	Mạng đầy đủ (full-mesh)	Vòng logic (ring)	Cây logic tự xây dựng theo kết nối
Thông điệp bầu chọn song song	Có thể gây xung đột	Không xung đột, xử lý hợp nhất	Được xử lý bằng cấu trúc cây
Xác định tiến trình điều phối	Tiến trình có ID cao nhất	Tiến trình có ID cao nhất	Tiến trình "tốt nhất" (ID cao nhất theo phản hồi)
Khả năng mở rộng	Kém nếu hệ thống lớn	Tốt hơn bully	Rất tốt trong mạng không đồng nhất
Yêu cầu biết trước thông tin hệ thống	Phải biết tất cả tiến trình	Biết tiến trình kế tiếp	Không cần biết các tiến trình khác

2. Ưu điểm và nhược điểm từng giải thuật

Bully Algorithm

- **Ưu điểm:**
 - Dễ cài đặt trong môi trường ổn định.
 - Nhanh nếu tiến trình có ID cao nhất vẫn hoạt động.
- **Nhược điểm:**
 - **Sử dụng nhiều thông điệp ($O(n^2)$)**, gây quá tải khi hệ thống lớn.
 - **Không xử lý tốt khi điều phối viên sống lại**, có thể dẫn đến **tranh chấp điều phối viên**.

- Phụ thuộc mạng đầy đủ (mọi tiến trình liên lạc được với tiến trình có ID cao hơn).

Ring Algorithm

- **Ưu điểm:**
 - **Thông điệp ít hơn** Bully ($O(n)$).
 - **Không gây xung đột**, xử lý song song tốt hơn.
 - Không cần biết toàn bộ hệ thống, chỉ cần biết “người hàng xóm kế tiếp”.
- **Nhược điểm:**
 - **Chậm hơn** trong hệ thống lớn vì thông điệp đi tuần tự.
 - **Phụ thuộc cấu trúc vòng**, nếu tiến trình trong vòng chết nhiều cần cơ chế dò lỗi thay thế.

Giải thuật Vasudevan 2004 (Không dây)

- **Ưu điểm:**
 - **Phù hợp với môi trường mạng không ổn định**, nhiều nhiễu, **không cần cấu trúc cố định**.
 - Tự động hình thành cây truyền thông hợp lý.
 - **Không yêu cầu biết danh sách tiến trình** – chỉ cần gửi đến láng giềng.
 - Có khả năng **hạn chế vòng lặp và mất gói tin**.
- **Nhược điểm:**
 - Cần xử lý tốt cơ chế xây dựng cây và tránh trùng lặp thông điệp.
 - **Phức tạp trong cài đặt hơn** so với Bully và Ring.

e. 1. Đề xuất kiến trúc bầu chọn để cân bằng giữa độ tin cậy, hiệu năng và chi phí giao tiếp

Kiến trúc đề xuất:

Kết hợp hai phương pháp: “chia nhóm – đại diện” và “thẻ bài với lực đẩy” theo hai tầng.

Tầng 1 – Chia nhóm – Đại diện (Cluster-Based Election):

- Hệ thống được chia thành nhiều **nhóm con (cluster)** dựa trên định danh hoặc vị trí địa lý/mạng.
- Mỗi nhóm tiến hành bầu chọn một **tiền trình đại diện (coordinator)**.
- Việc bầu chọn trong nhóm sử dụng quy tắc đơn giản (ví dụ: chọn tiến trình có ID nhỏ nhất, hoặc dùng Bully/RAFT nội bộ nhóm).
- **Ưu điểm:**
 - Giảm chi phí giao tiếp, vì chỉ cần truyền thông trong nhóm.
 - Bầu chọn cục bộ nhanh, dễ quản lý, đảm bảo độ tin cậy cao trong nhóm.

Tầng 2 – Phân phối đều đại diện bằng thẻ bài – lực đẩy (Token Repulsion):

- Các đại diện được xem là “**thẻ bài**” phân phối trong không gian logic (mạng lưới hệ thống).
- Mỗi đại diện phát lực đẩy để giữ khoảng cách đều nhau, đảm bảo không tập trung quá nhiều đại diện ở một vùng.
- Nếu phát hiện một đại diện quá gần với các đại diện khác (dựa vào các “tổng lực”), hệ thống tự động thay thế bằng tiến trình khác ở vùng thưa đại diện hơn.
- **Ưu điểm:**
 - Đảm bảo **phân bố đều**, tăng khả năng bao phủ và độ tin cậy toàn hệ thống.
 - Tự động **thích nghi** khi có tiến trình rớt/hỏng, giảm thiểu việc bầu chọn lại toàn hệ thống.

2. Giải thích lý do chọn giải pháp đó

Lý do chọn kiến trúc kết hợp:

Tiêu chí	Lý do
Độ tin cậy	Việc chia nhóm đảm bảo rằng sự cố cục bộ (ví dụ một node hỏng) không ảnh hưởng toàn hệ thống. Lực đẩy giúp hệ thống tự điều chỉnh khi mất đại diện.
Hiệu năng	Bầu chọn nội nhóm nhanh vì chỉ cần trao đổi giữa số lượng nhỏ node. Việc truyền lực đẩy có thể lan truyền bất đồng bộ , không ảnh hưởng hiệu năng chính.

Chi phí giao tiếp	Chỉ các tiến trình đại diện cần liên lạc giữa nhóm, giảm $O(n^2)$ xuống $O(g^2)$ với g là số nhóm. Việc cập nhật lực đẩy có thể thực hiện có giới hạn phạm vi .
Khả năng mở rộng	Phù hợp với hệ thống hàng ngàn node , do cả hai tầng đều cho phép mở rộng tuyến tính.
Tự điều chỉnh – Tự phục hồi	Lực đẩy giúp hệ thống phân bố lại đại diện mà không cần bầu lại toàn bộ, phù hợp với môi trường động như cloud hoặc P2P.